Big-Vul: Analysis of Vulnerability Patterns and Machine Learning for Software Security

An Empirical Study of C/C++ Software Vulnerabilities and Their Predictability from Repository and Code-Change Metadata

TECHNICAL ANALYSIS REPORT

Prepared by	Muhammad Ali
Institution	National Textile University, Faisalabad
Dataset	Big-Vul (all_c_cpp_release2_0.csv)

Abstract

This report performs empirical analysis of the Big-Vul dataset, a large dataset comprising of C/C++ software vulnerabilities along with the information of the respective fix-commits on GitHub. The distribution of categories and CWE weaknesses of the vulnerabilities, the concentration of vulnerabilities within open-source repositories and the years of publication and the relationship between the size of the fix-commit made and the CVSS score of the vulnerability are analyzed. A multiclass Logistic Regression model is subsequently trained for predicting the coarse-grained vulnerability_classification from the features of the vulnerability and code change. The model yielded 20.8% accuracy (macro-F1 = 0.170, weighted-F1 = 0.222) on more than 20 highly imbalanced classes a result presented here as an honest baseline and not as an actual detector, given the extreme granularity of the classes as shown in Section 5 of the paper.

Research Questions

- Which categories of vulnerabilities and CWE types are most common in the dataset?
- How are vulnerabilities distributed across projects and publication years?
- Does the severity of vulnerabilities correlate with the size of the code fix?
- Does information about vulnerability and features of code change determine coarse vulnerability classification?

Alignment with Research Objectives

The current work represents the second empirical study in the portfolio of the author, coming after a previous study on developer productivity and code quality with AI assistance (session-level data, self-reported). While the previous study focused on the tradeoffs on the productivity of individual developers, the current analysis focuses on empirical repository data on how vulnerabilities are realized and propagated in 348 real-world C/C++ open-source projects. As such, both studies show the progress from a controlled behavior-based dataset to a real-world repository-based one. Importantly, this project directly relates to the author's interest in Mitacs Globalink Research Internship placements in terms of Coding Agent Impact on Repository Quality and Secure LLM-Based Code Generation, with the current project being just a small-scale realization of the same methodology (repository mining, defect/vulnerability classification, feature importance analysis).

Data and Preprocessing

This data set is sourced from the Big-Vul dataset accompanying the MSR 2020 conference paper titled "A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries" (Fan et al., 2020). Each row contains the relationship between a CVE-commit and its metadata (CVSS metrics, CWE categories, affected projects) and code-change attributes.

Metric	Value
Raw records	4,432
Records after removing invalid commit IDs ("#F0")	4,430
Distinct projects	348 (Chrome alone: 1,518 records, 34%)
Distinct CWE categories	91
CVSS score — mean / median	5.95 / 6.60 (0–10 scale)
Publication years covered	2009–2019 (peak: 2017, 1,013 records)

Handling of data quality issues: The two fields called authentication_required and access_complexity had 358 missing values for each; availability_impact, confidentiality_impact, and integrity_impact had 1,145, 1,885, and 1,998 missing values respectively – all were kept in the "missing" class and not imputed, as CVSS sub-scores are often not evaluated for the old CVE entries but not missing at random. The two observations containing the placeholder value for commit_id field (#F0) were dropped from the dataset.

Vulnerability Distribution

There is an uneven distribution of CWE categories in the dataset, where CWE-119 (Buffer Overflow) has 694 records (15.7%). This is followed by CWE-20 (Improper Input Validation) with 445, CWE-125 (Out-of-Bounds Read) with 365, and CWE-200 (Information Exposure) with 272 records respectively.

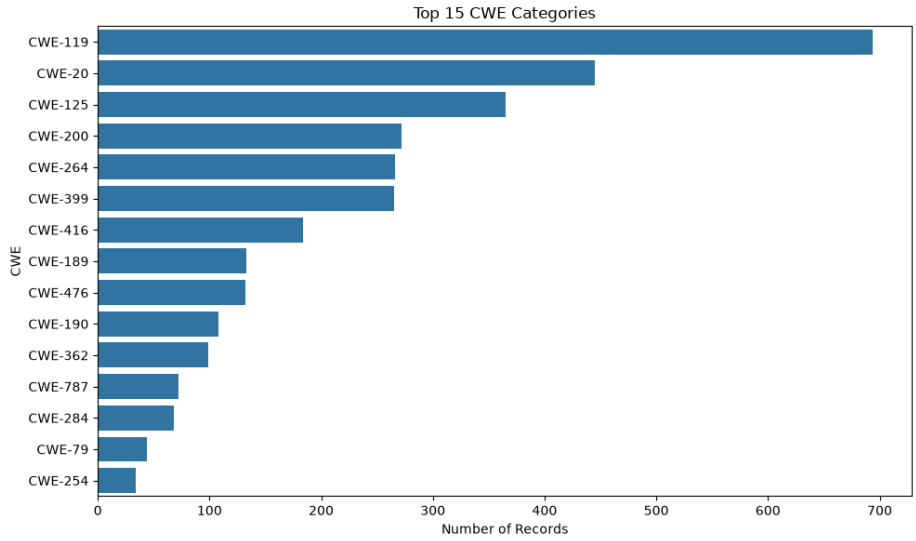


Figure 1. Top 15 CWE categories by record count.

Vulnerability classification labels that use a courser graining follow the same trend on a larger scale: "DoS" (denial of service) accounts for the highest number of classes (1,159 samples), while combined classes such as "DoS Overflow" (380) and "+Info" (information disclosure, 306)

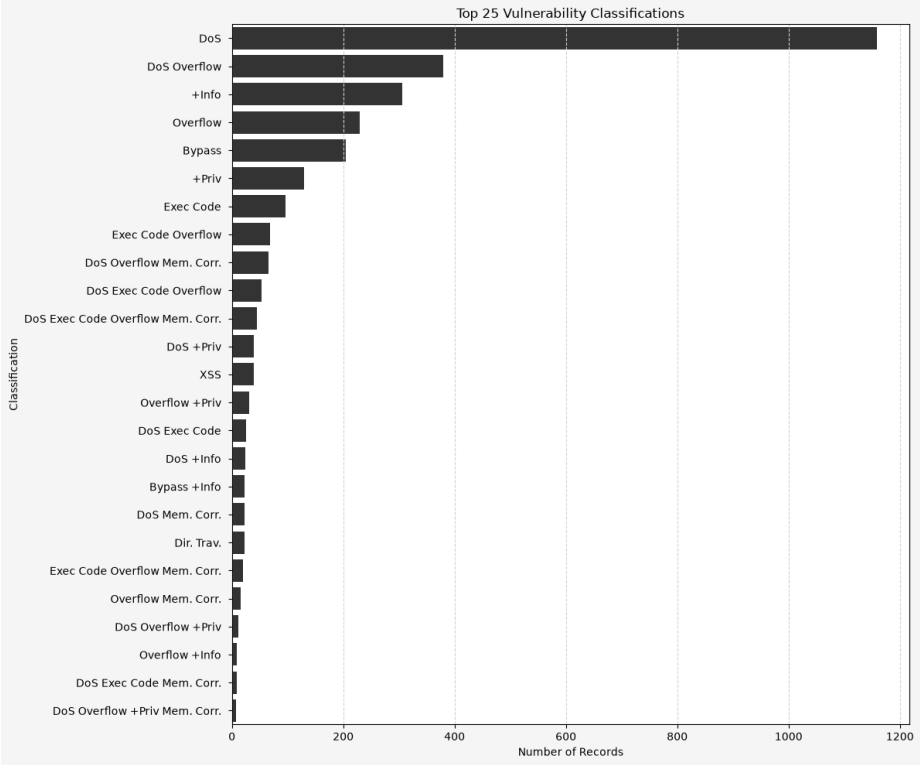


Figure 2. Top 25 vulnerability_classification categories by record count.

Disclosures of vulnerabilities within the database increase dramatically from 2015 onwards and peak at 1,013 entries in 2017, before dropping off to 2019 – an indication of actual growth in CVE activities as well as the set end date of 2019 for the data collection.

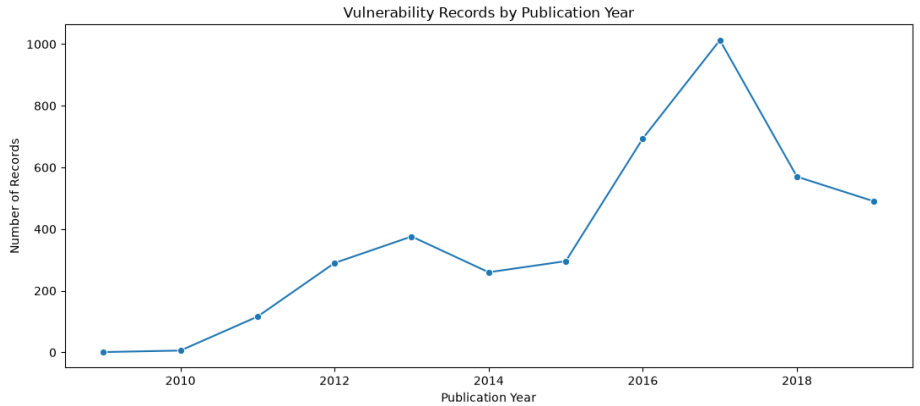


Figure 3. Number of vulnerability records by publication year (2009–2019).

Severity Analysis

The CVSS scores are distributed between 0 and 10, with an average value of 5.95 and a median value of 6.60. The left-skewed nature of the distribution is clearly seen when looking at the 25th and 75th percentiles.

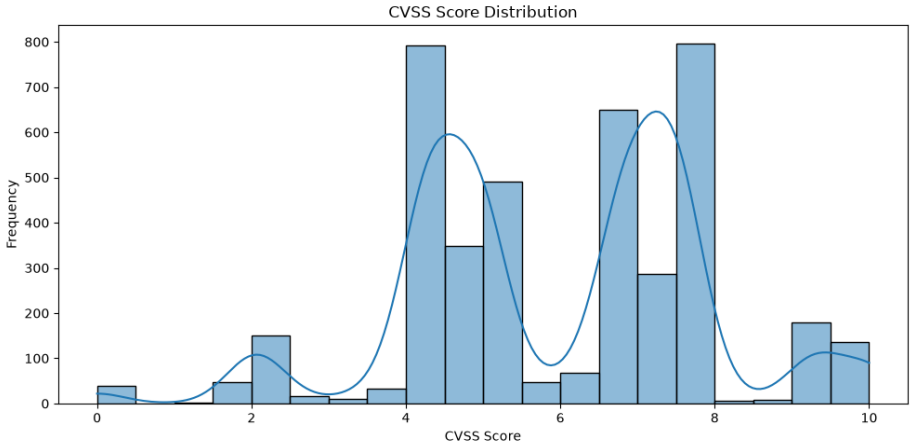


Figure 4. Distribution of CVSS severity scores across all records.

There is substantial variation in severity ratings among vulnerability categories, with vulnerability categories relating to code execution and memory corruption typically rated more severe than those of information disclosure or bypass categories.

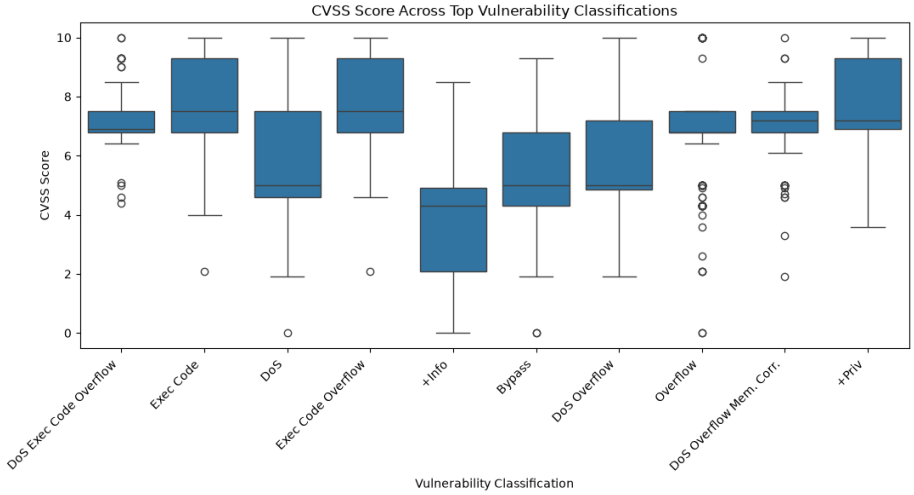


Figure 5. CVSS score distribution across the most frequent vulnerability classification categories.

Project Concentration

The vulnerability data is highly skewed towards few highly visible open source projects, such as Chrome (1,518), Linux (927), Android (374), and ImageMagick (189). It is essential to keep this in mind while analyzing trends across the whole dataset, because the data set cannot be considered as representative for all software projects – only some of the highly visible projects are covered by it.

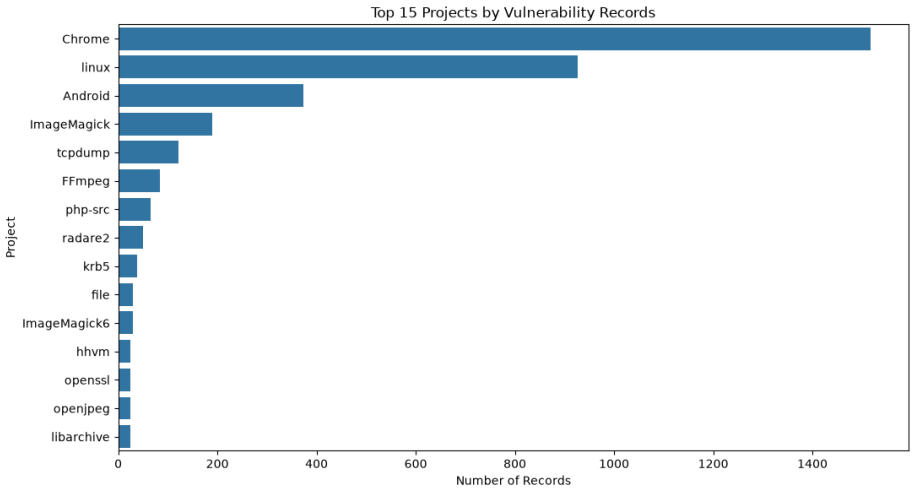


Figure 6. Top 15 projects by number of recorded vulnerabilities.

Patch Size and Severity

The total number of lines modified per fix (additions+deletions) is highly right-skewed as well: median 0, mean 6.71, with the largest being 480 lines modified in one fix.

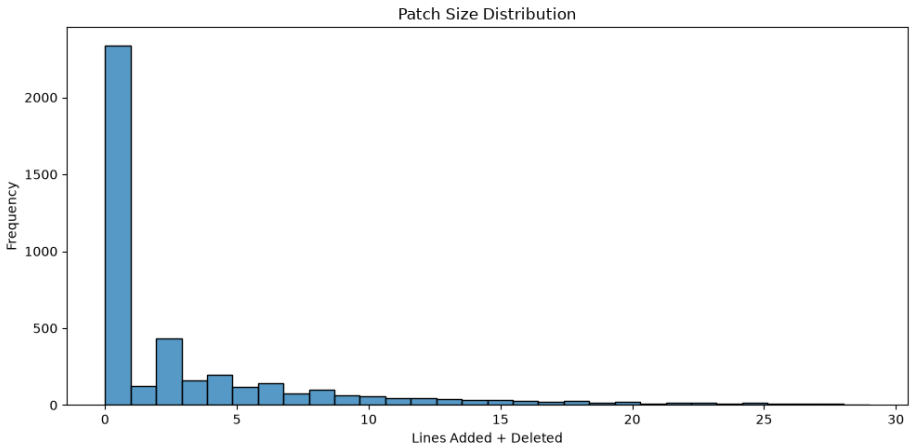


Figure 7. Distribution of lines changed per vulnerability-fixing commit (95th-percentile trimmed).

The result of the Spearman's correlation between the number of changes and the CVSS score yielded $\rho = -0.144$ ($p < 0.001$), which denotes a statistically significant but a weak negative correlation, suggesting that patches of smaller sizes are minimally associated with vulnerabilities of higher severity. Due to the minuscule size of the effect, this must be interpreted as an association, not causation.

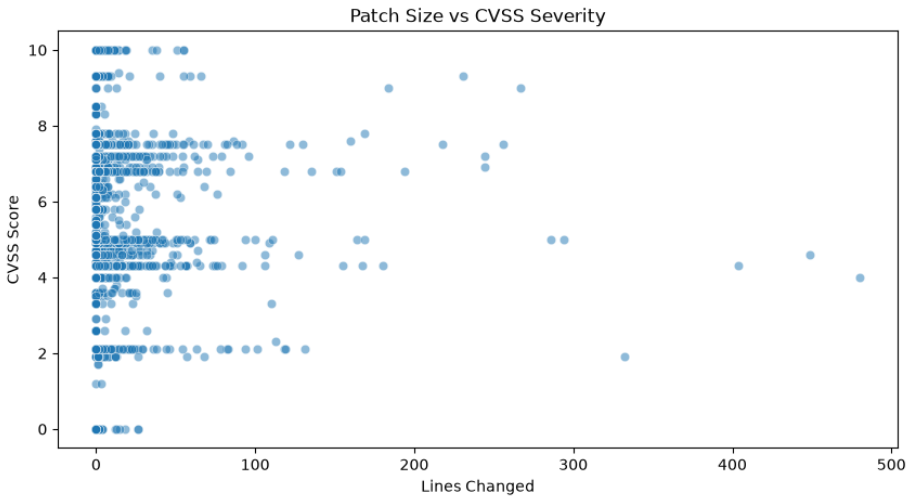


Figure 8. Relationship between lines changed and CVSS score.

Predictive Modeling

Methodology

Multiclass Logistic Regression model was developed for predicting the vulnerability_classification feature using nine features that represent vulnerability severity/impact (score, access_complexity, authentication_required, confidentiality/integrity/availability_impact), repository features (project, obtained through preprocessing pipeline) and code change features (lines_changed). Classes that had less than 20 observations were collapsed into an "Other" class. Observations where the target feature was not defined (1,319 out of 4,430) were removed from the analysis. This resulted in 3,111 observation dataset (70%/30% train/test split). Numerical features were imputed using median while categorical ones – imputed and then one-hot encoded.

Results

Metric	Value
Overall accuracy	20.8%
Macro-F1	0.170
Weighted-F1	0.222

The performance of this system is extremely different across classes; for example, some classes that have representation, such as "DoS", had a reasonable precision rate of 0.77 but a very poor recall rate of 0.16, showing that the classifier is being conservative about predicting the majority class, whereas several underrepresented classes, including "DoS Mem. Corr." and "DoS Exec Code Overflow", had scores of 0.00 in all the measures this was due to the 20+ class imbalance.

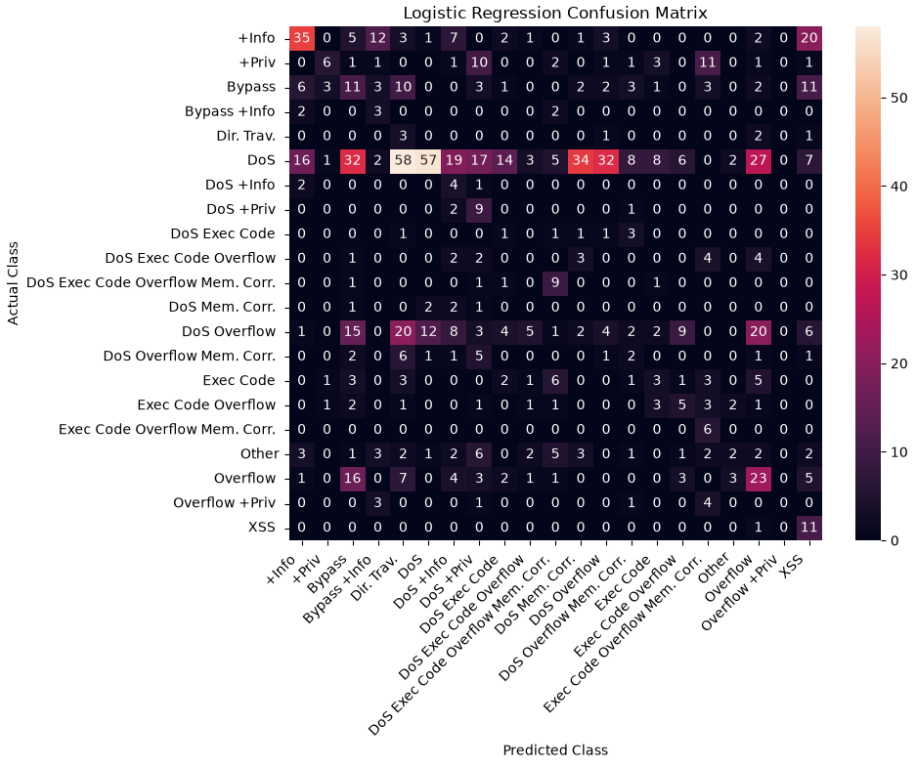


Figure 9. Confusion matrix for the multiclass vulnerability-classification model.

The measure of feature importance, defined as the mean absolute coefficient of all the classes, is provided below. The severity-based features based on CVSS far outweigh the others in terms of influencing model decisions. The measure of feature importance, defined as the mean absolute coefficient of all the classes, is provided below. The severity-based features based on CVSS far outweigh the others in terms of influencing model decisions.

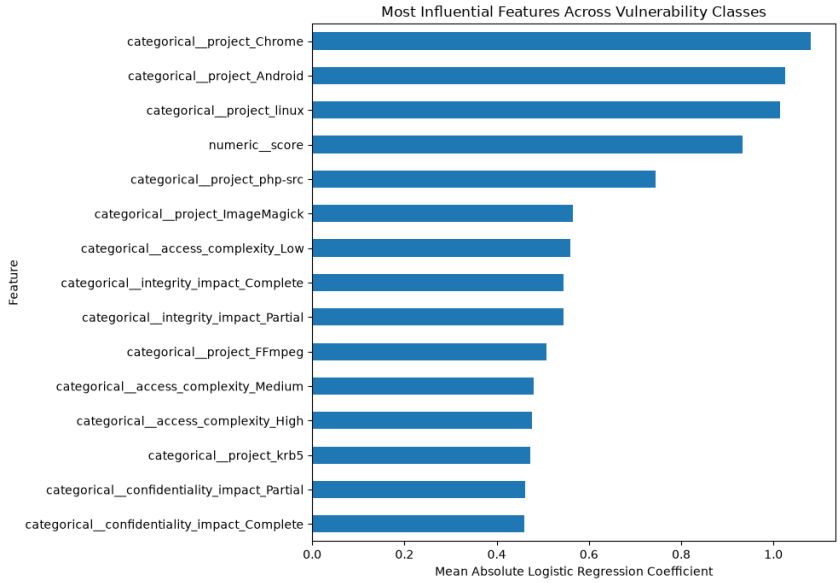


Figure 10. Mean absolute coefficient magnitude by feature (Logistic Regression).

Discussion and Limitations

- This data set includes the vulnerabilities that have been identified and made public, along with corresponding fixing commits. It is not a random sample of all vulnerabilities in software programs; therefore, these rates cannot be generalized to "how vulnerable is software in general."
- The problem of class imbalance is acute (91 types of CWE; 20+ vulnerability_classification labels), and it adversely impacts the performance of multiclass classifiers irrespective of the algorithm used.
- It is an old dataset showing C/C++ vulnerability trends from 2009 to 2019 and not current trends of 2026.
- Concentration of projects (34% of the records from Chrome alone) implies that the learned relations can be more characteristic for resource-rich projects than for software development in general.
- The Logistic Regression model is used as a reproducible baseline and not for the deployment of a vulnerability detection system since the accuracy rate of 20.8% should be seen together with the macro-F1 of 0.170.

Conclusion

This paper shows how it is possible to use machine learning with software repository data to analyze the attributes, severity, and code fixing of vulnerabilities. From the CWE and vulnerability_classification distributions, it is clear that a few weakness types account for the vast majority of the vulnerabilities found in reality; there is a significant but weak association between severity and patch size; and the vulnerability category is partially predictable based on metadata and code fix information alone, an observation which agrees with the extremely fine granularity of classes seen above. This paper establishes a replicable baseline for future studies in empirical software engineering and vulnerability analysis, and extends the work of the author on AI-based developer productivity.

References

- Fan, J., Li, Y., Wang, S., & Nguyen, T. N. (2020). A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. Proceedings of the 17th International Conference on Mining Software Repositories (MSR 2020), 508–512. https://doi.org/10.1145/3379597.3387501
- MITRE Corporation. (n.d.). Common Weakness Enumeration (CWE). Retrieved 2026, from https://cwe.mitre.org
- FIRST.org, Inc. (n.d.). Common Vulnerability Scoring System (CVSS). Retrieved 2026, from https://www.first.org/cvss
- National Institute of Standards and Technology. (n.d.). National Vulnerability Database (NVD). U.S. Department of Commerce. https://nvd.nist.gov